

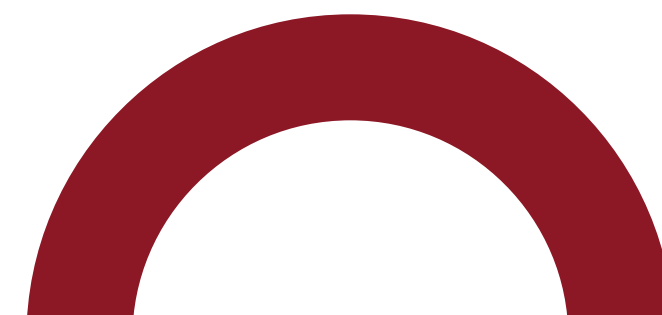


2026-1 BASIC STUDY ML 6주차

Tree Based Models

SESSION LEADER: 20기 박준희

2026.02.12



과제 우수자 발표

윤지원 님

화면 공유해주시고
5분 이내로 설명 부탁드립니다 :)



Decision Tree

Keywords

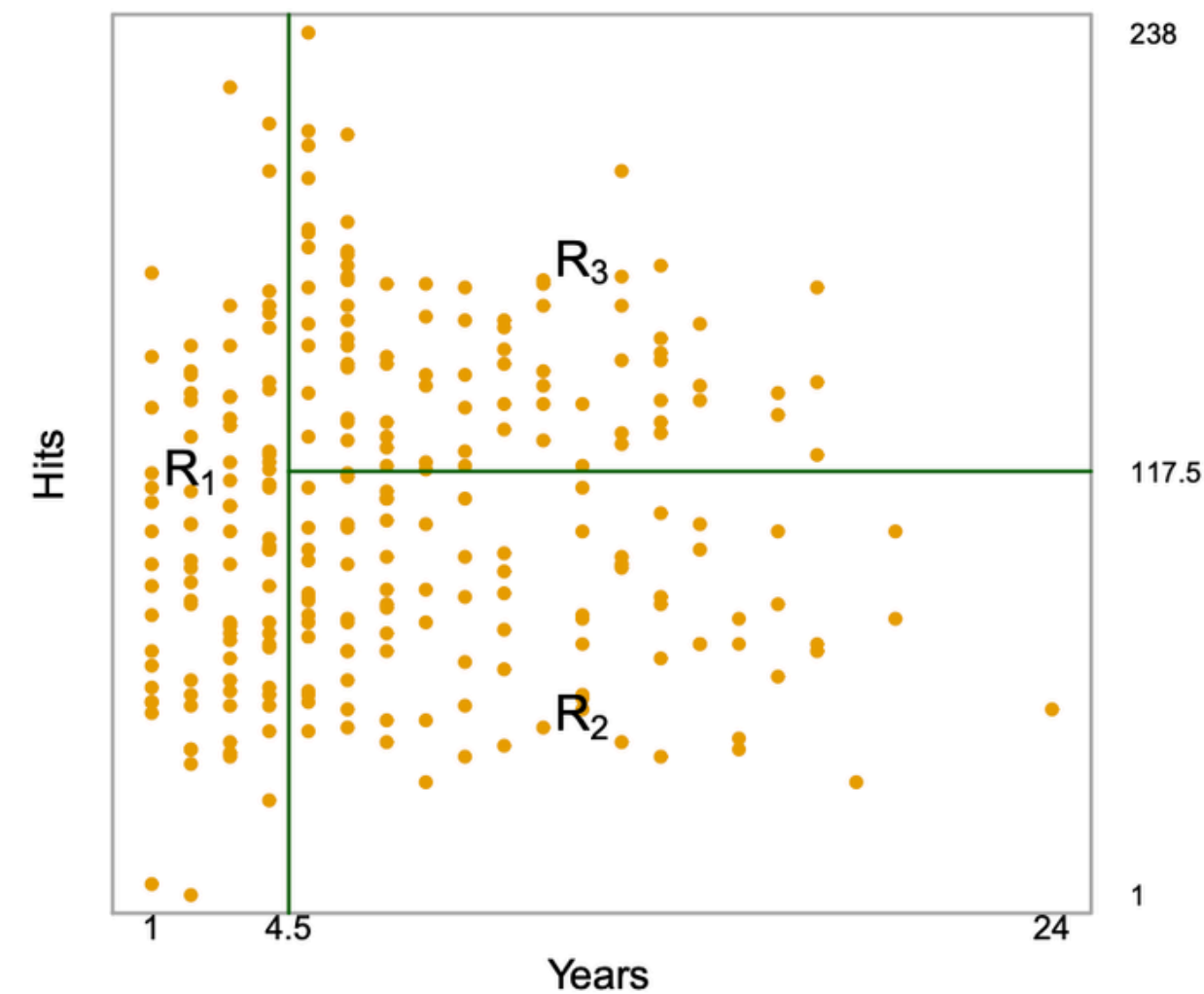
Introduction

Decision Tree Algorithm

Decision Tree

Hitters Data: Predict Salary of MLB Players based on Year and Hits.

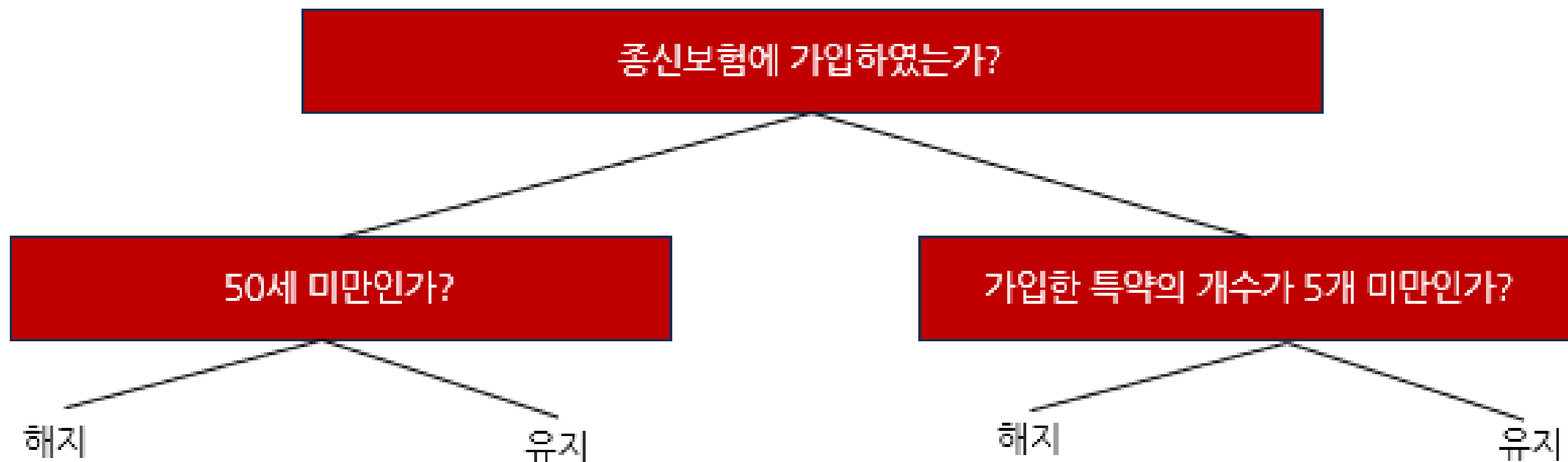
(a) Partitioned Predictor Space



(b) Fitted Model



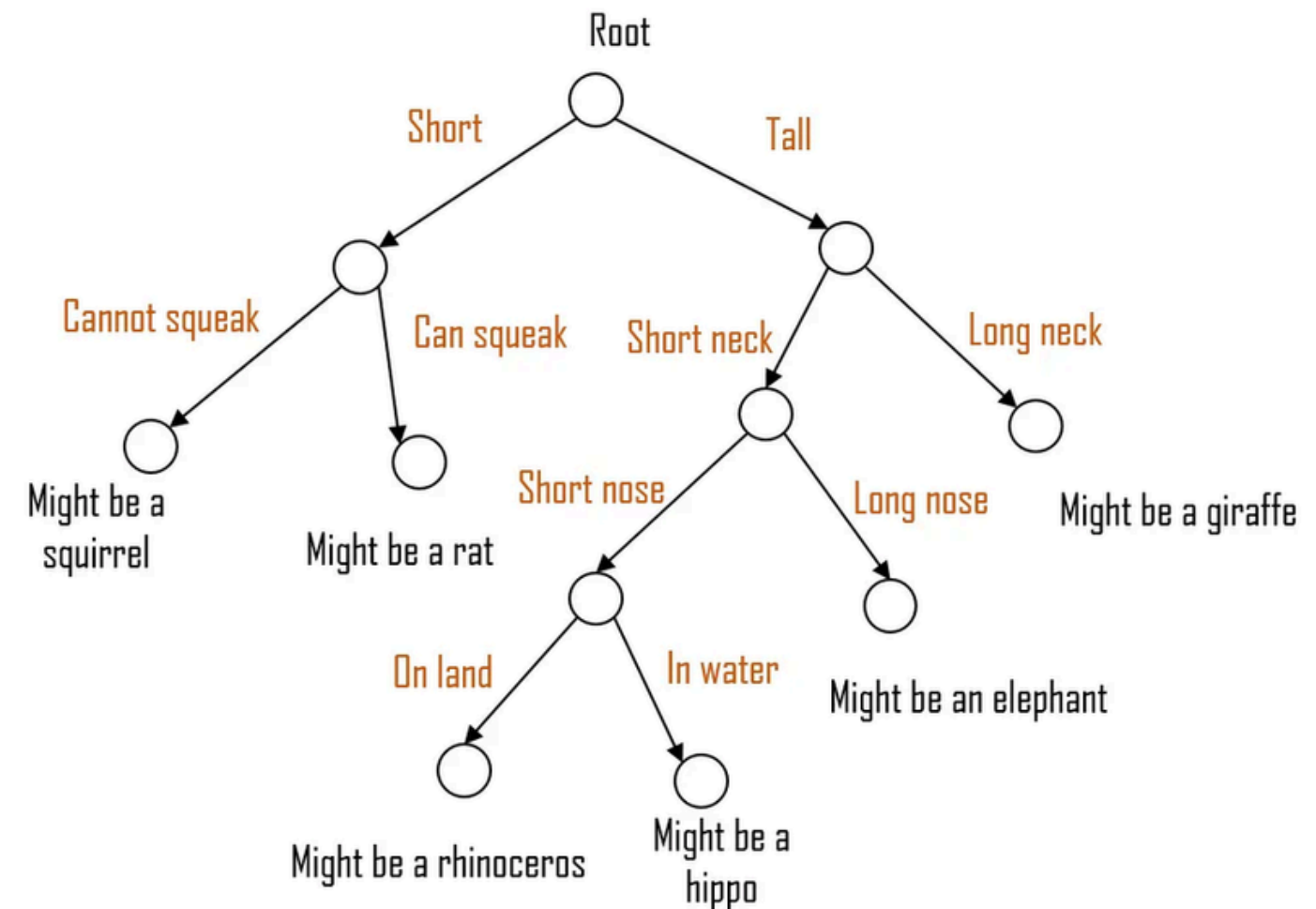
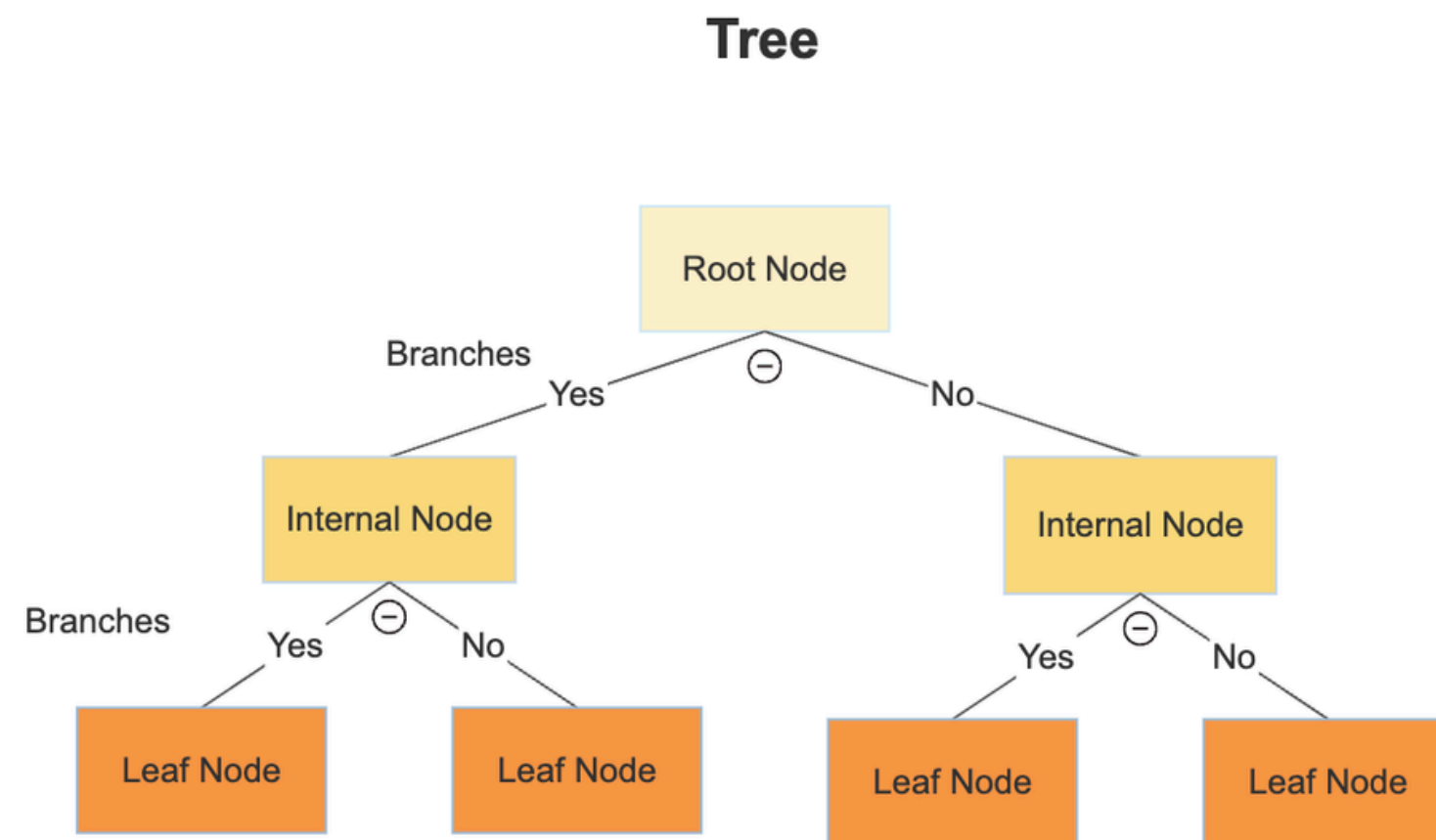
Decision Tree



- “Decision Tree” → 나무처럼 구성된 표
- 위에서부터 출발하여 아래쪽까지 도달하는 방향으로 해석
- 각 질문(“노드”)에 대한 응답으로 데이터 분할

Tree Structure

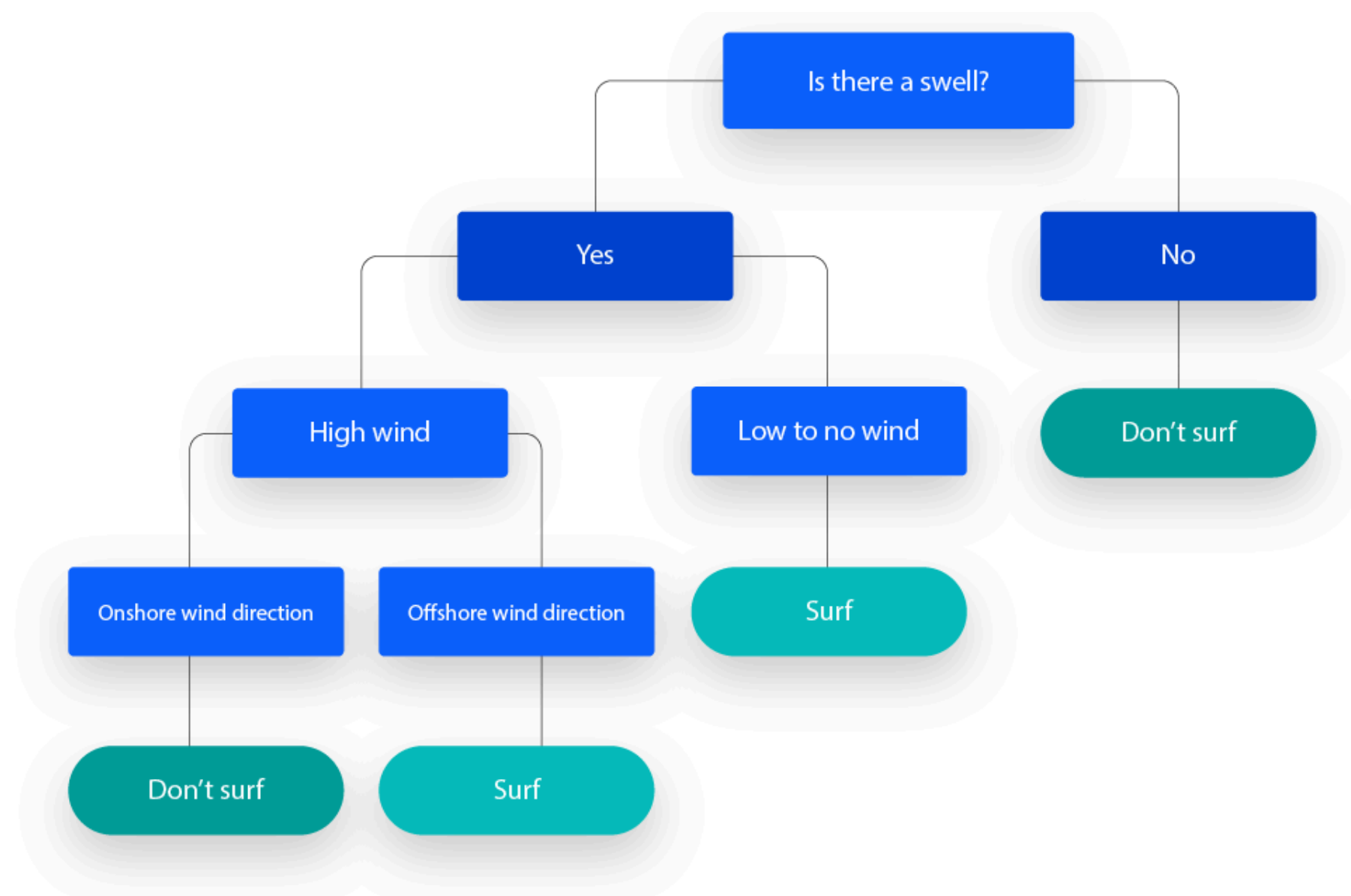
- Root Node : 데이터를 처음 나누는 기준이 되는 노드
- Internal(Decision) Node : 데이터의 특정 feature에 따라 분기(split)
- Leaf(Terminal) Node : 최종 예측값이 결정되는 노드



Decision Tree

나무의 Root Node에서 출발하여 Leaf Node에 이르기까지의 분기를 수행

→ **분류 성능은 분기의 기준에 달려있는데, 분기의 기준은 어떻게 결정할까?**



Choosing the Best Split

- **Recursive Binary Splitting (순차적 분할)**

- A good partition yields similar response values in a same node and different values in different nodes
- 각 영역의 Information gain을 증가시키는 방향으로 학습!

→ 순도(homogeneity)가 높을수록, 불순도(Impurity)가 낮을수록 좋음

- **Impurity(불순도)**

- Regression: Residual Sum of Squares (RSS)
- Classification: Gini Index, Entropy

→ Sequentially detect the partition that minimizes total impurities

Impurity

- **Impurity(불순도):**

- 트리 전체의 불순도는 각 노드의 불순도를 데이터 수로 가중평균하여 계산

- **Regression : Residual Sum of Squares (RSS)**

- 회귀 트리는 각 노드에서 평균값을 예측값으로 사용
- RSS를 최소화하는 방향으로 분할

- **Classification**

- Gini Index : $1 - (\text{각 항목이 차지하는 비율의 제곱 합})$
 - 불순도가 낮을수록 더 낮은 분할
- Entropy
 - Information Gain = 전체 엔트로피 - 분할 후 엔트로피
 - 엔트로피(불확실성)가 작을수록 데이터가 순수한 상태

→ **Sequentially detect the partition that minimizes total impurities**

Recursive Binary Splitting

- Regression Tree

ex Toy Example: Regression Tree

x	1	2	3	4
y	0	2	10	12

$$1. \ x \leq 1 \text{ vs } x > 1: (0 - 0)^2 + (2 - 8)^2 + (10 - 8)^2 + (12 - 8)^2 = 56$$

$$2. \ x \leq 2 \text{ vs } x > 2: (0 - 1)^2 + (2 - 1)^2 + (10 - 11)^2 + (12 - 11)^2 = 4$$

$$3. \ x \leq 3 \text{ vs } x > 3: (0 - 4)^2 + (2 - 4)^2 + (10 - 4)^2 + (12 - 12)^2 = 56$$

Recursive Binary Splitting

- **Classification Tree**

ex Toy Example: Binary Classification

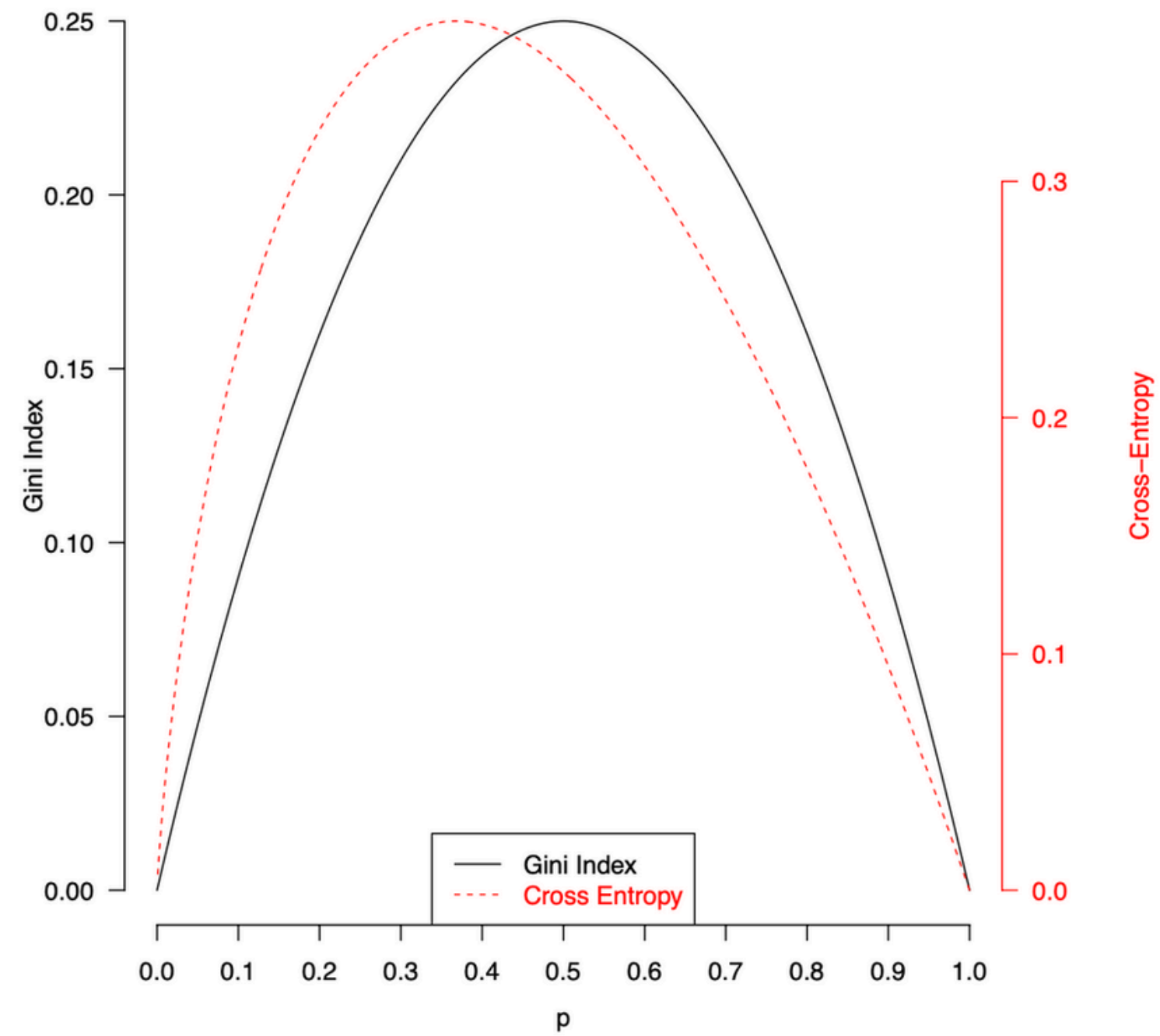
x	1	2	3	4
y	1	1	1	2

$$1. \quad x \leq 1 \text{ vs } x > 1: 1 \cdot 0 + 0 \cdot 1 + \frac{2}{3} \cdot \frac{1}{3} + \frac{1}{3} \cdot \frac{2}{3} = \frac{2}{3}$$

$$2. \quad x \leq 2 \text{ vs } x > 2: 1 \cdot 0 + 0 \cdot 1 + \frac{1}{2} \cdot \frac{1}{2} + \frac{1}{2} \cdot \frac{1}{2} = \frac{1}{2}$$

$$3. \quad x \leq 3 \text{ vs } x > 3: 1 \cdot 0 + 0 \cdot 1 + 0 \cdot 1 + 1 \cdot 0 = 0$$

Impurity



Decision Tree

- 불순도가 낮아지는 방향으로 주어진 데이터를 분류하는 분석방법
- 어떤 불순도 지표를 택할지, 어떻게 분류할지에 따라 생성 방식 상이
- **CART**(Classification and Regression Tree)
 - 불순도 : Gini Index 사용, Binary Classification, Regression/Classification, Pruning
- **CHAID**(Chi-squared Automatic Interaction Detection)
 - 불순도 : Chi-squared value 사용, Multi-class Classification, Categorical variable
- **C4.5**
 - 불순도 : Entropy 사용, Multi-Class Classification, Pruning

Pruning

- **Size of tree** is a tuning parameter.
 - Too large Tree: Overfitting (High Variance/Low Bias)
 - Too small Tree: Underfitting (Low Variance/High Bias)

Where to stop?

- Pruning : 트리의 크기를 제어하는 최적화 문제
 - pre-pruning: early stopping; maximum depth, minimum sample per leaf...
 - post-pruning: fit full tree and then cut; 학습 후에 불필요한 가지 제거
 - Objective Function: $\text{Total Impurity of } T + \lambda|T|$
 - λ is a tuning parameter
 - $\lambda = 0 \rightarrow T = \text{Full tree (overfitting)}$
 - $\lambda = \infty \rightarrow T = \text{Minimal tree or empty tree (underfitting)}$
 - select λ via cross-validation Pruning



Ensemble

Keywords

Introduction

Bagging

Boosting

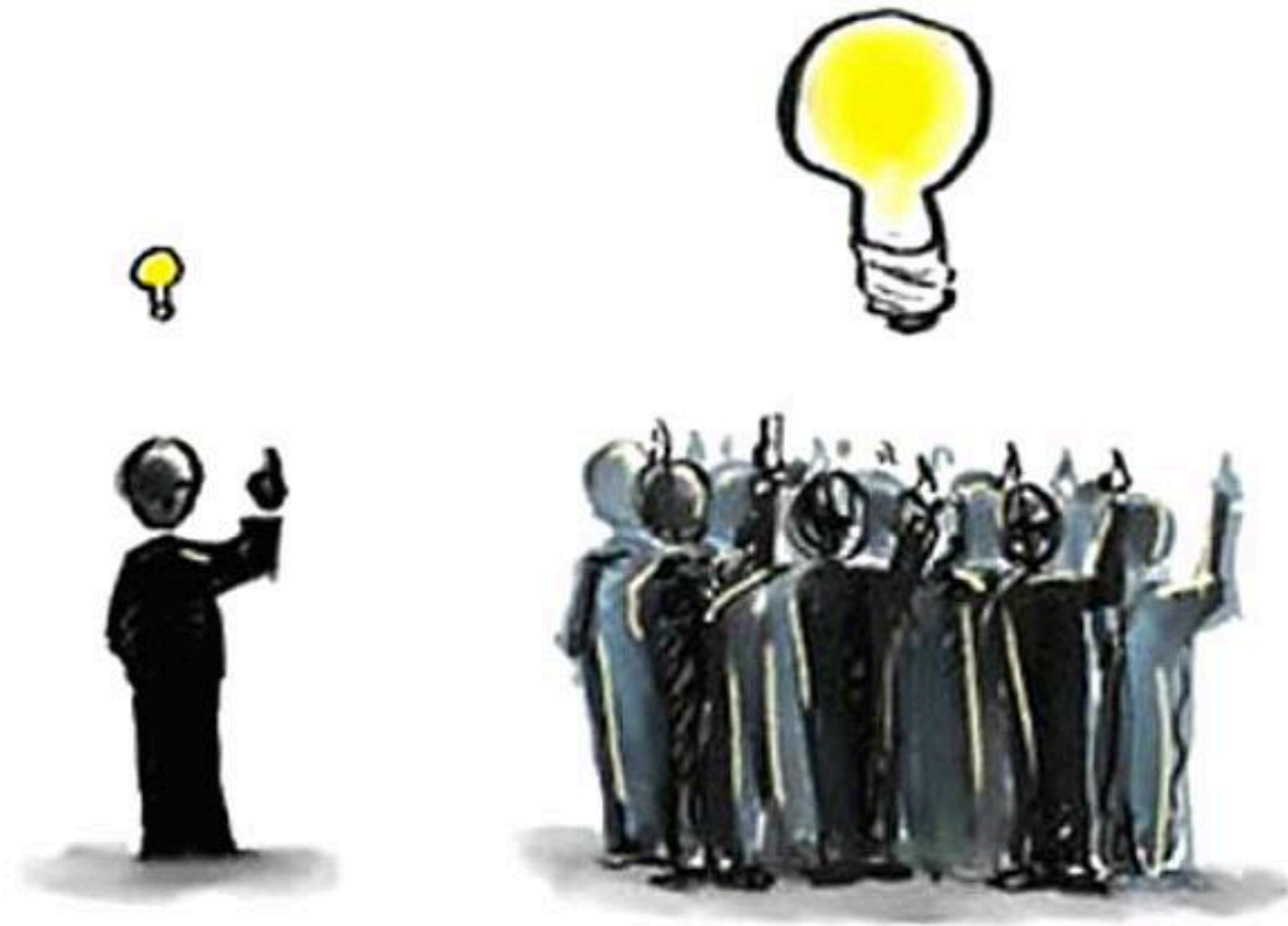
Stacking

Ensemble



여러 개의 악기가 조화롭게 연주하는 것

Ensemble



Wisdom of the crowd

Ensemble Learning

- **Tree is a weak learner with large variance.**
→ **Reduce the variance by repeated learning !**
- **Bootstrapping and then Fitting**
- **Ensemble is a popular idea in ML**
 - 다수의 기본 분류 모델 (base classifier, weak classifier)의 예측 결과를 종합
 - 정확한 예측 성능을 얻도록 하는 방법론

“Combine the outputs of many “weak” classifiers to produce a powerful committee!!”

Ensemble Learning

- **Ensemble Learning의 목적**

- Weak Learner(약한 모델)를 조합하여 Strong Learner(강한 모델) 생성
- **Bias-Variance Trade-Off**
 - Decision Tree → Low Bias, High Variance
 - Ensemble → Bias와 Variance를 조절하여 최적화!

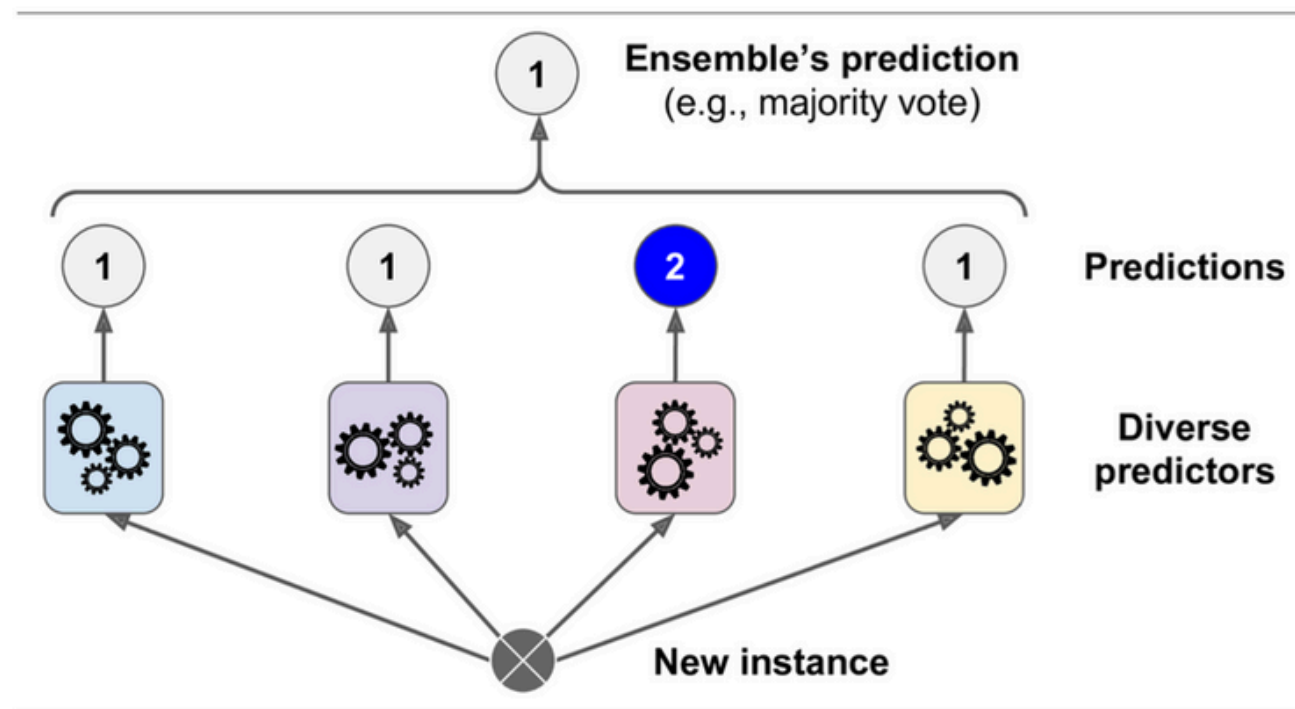
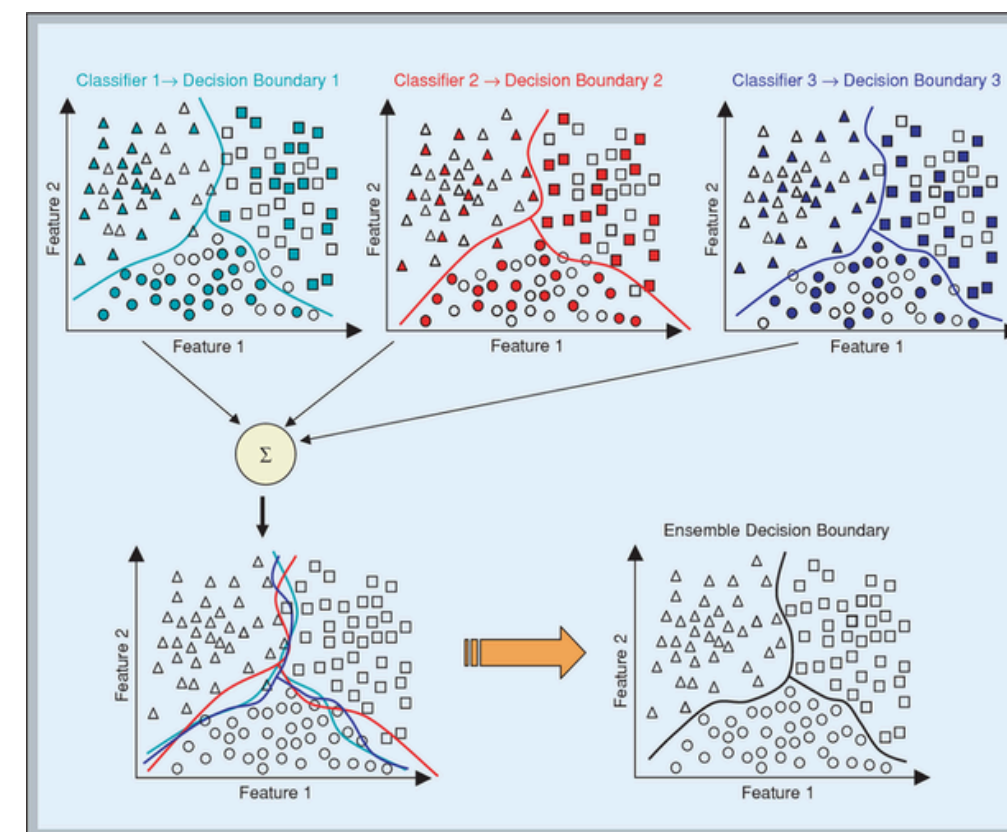


Figure 7-2. Hard voting classifier predictions



Ensemble Methods

Voting 다른 학생; 다른 알고리즘 사용 가능

- Hard Voting
- Soft Voting
- Weighted Voting

Stacking

- Meta Level Learning
- Blending

★ Bagging 같은 학생; 같은 알고리즘 사용

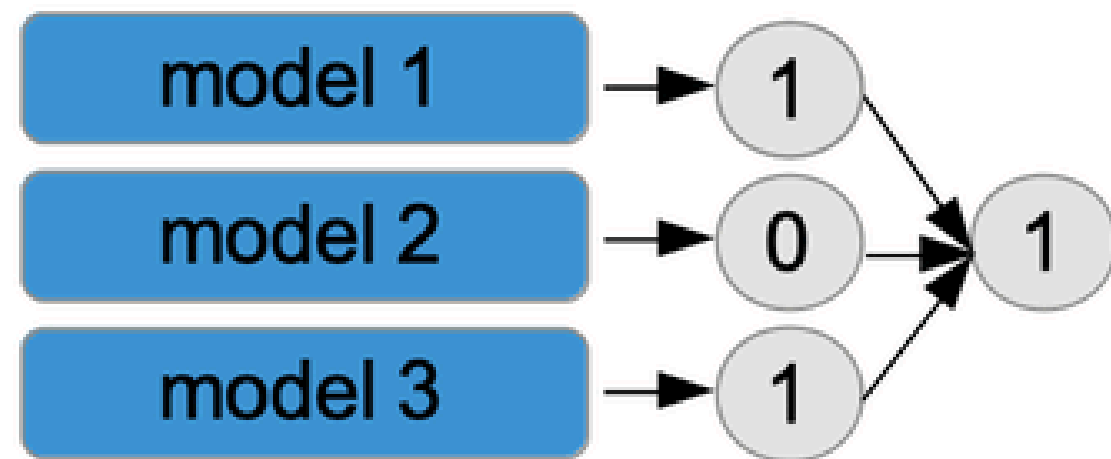
- Bootstrap
+ Aggregating

★ Boosting

- Error Learner

Voting

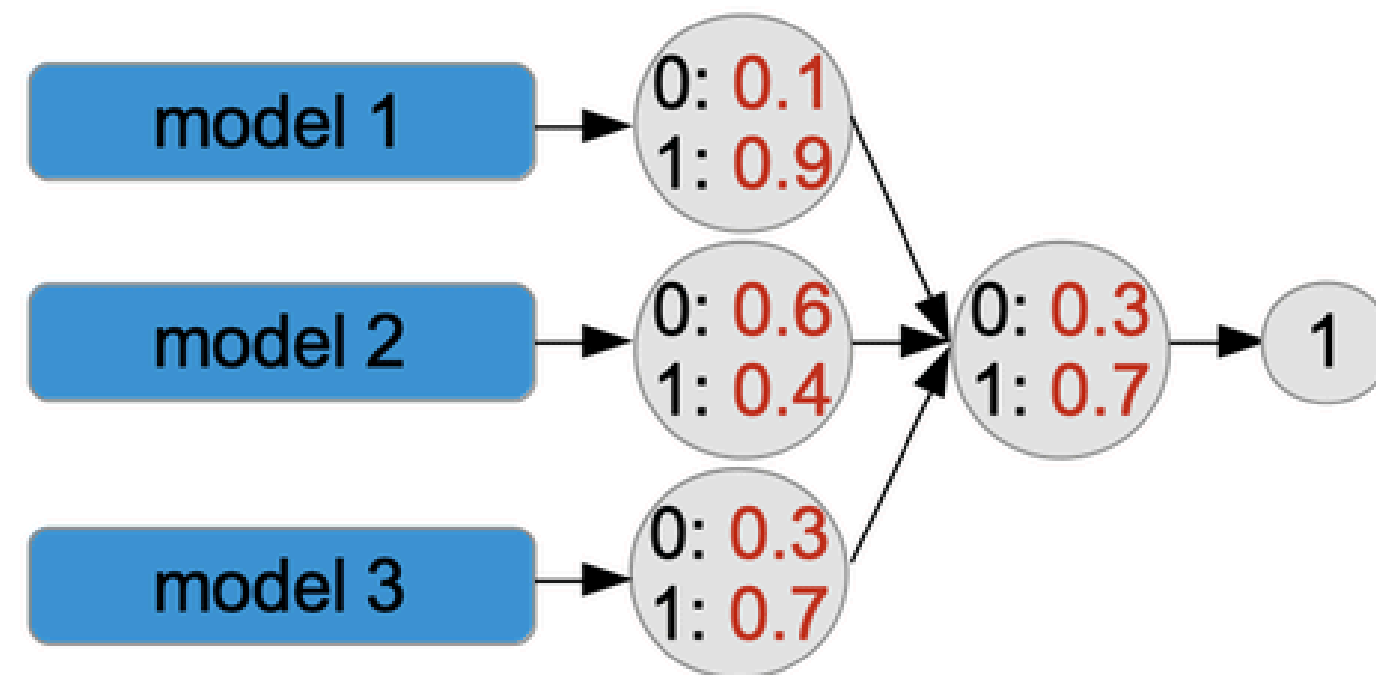
Hard Voting : Majority Voting



Hard voting

동점인 경우, 빠른 클래스를 따름

Soft Voting : Average Voting

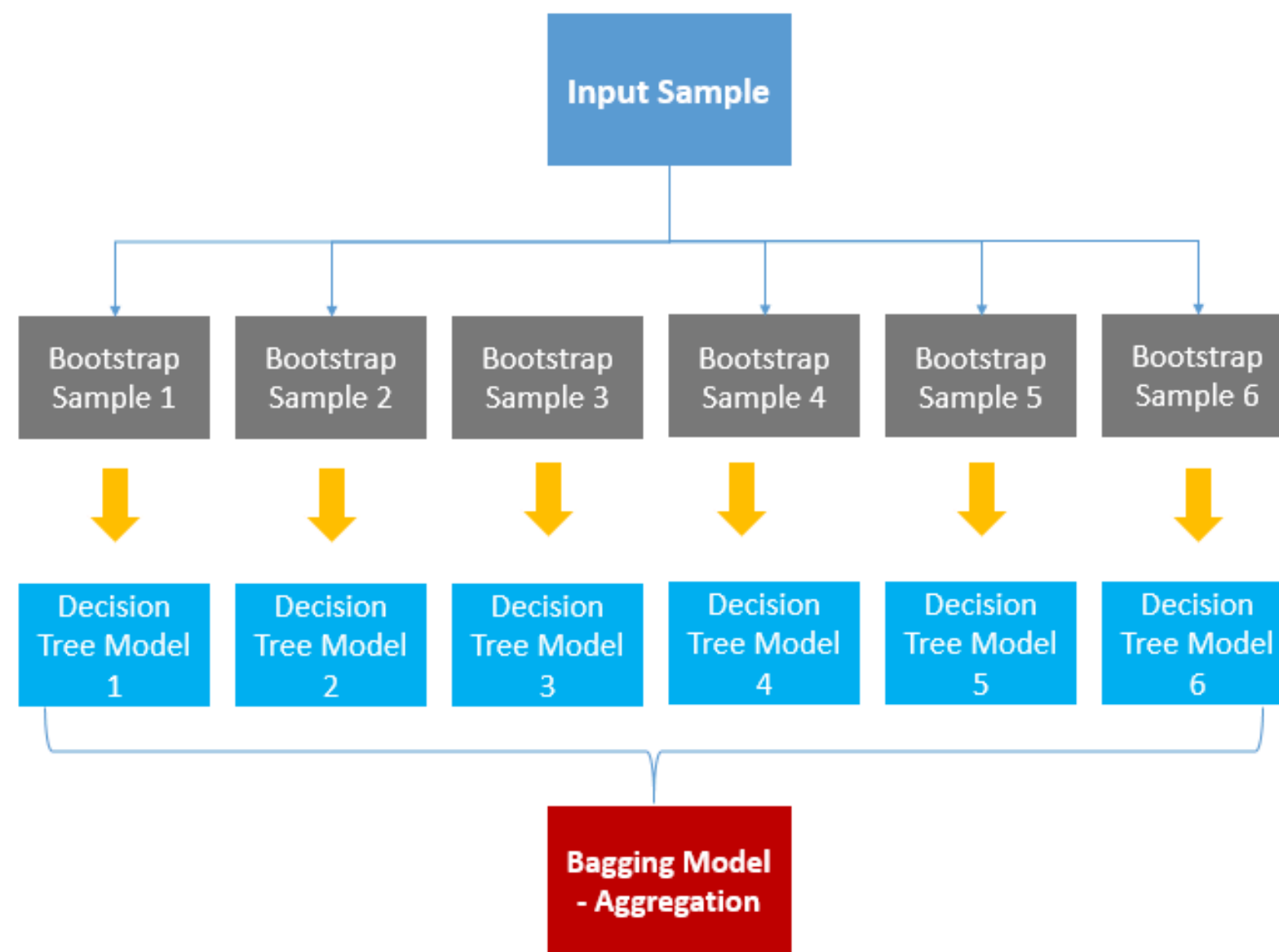


Soft voting

Simple/Weighted Average

Bagging

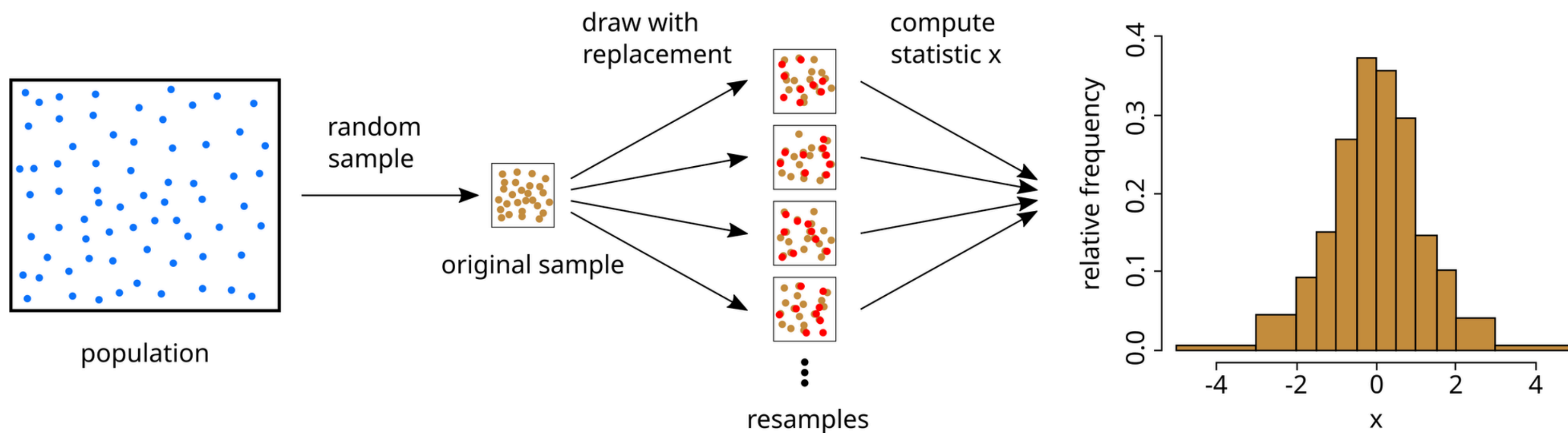
- **Bagging = Bootstrap + Aggregating(Average)**
- Bootstrap : sampling with Replacement → Variance 개선



문제은행!

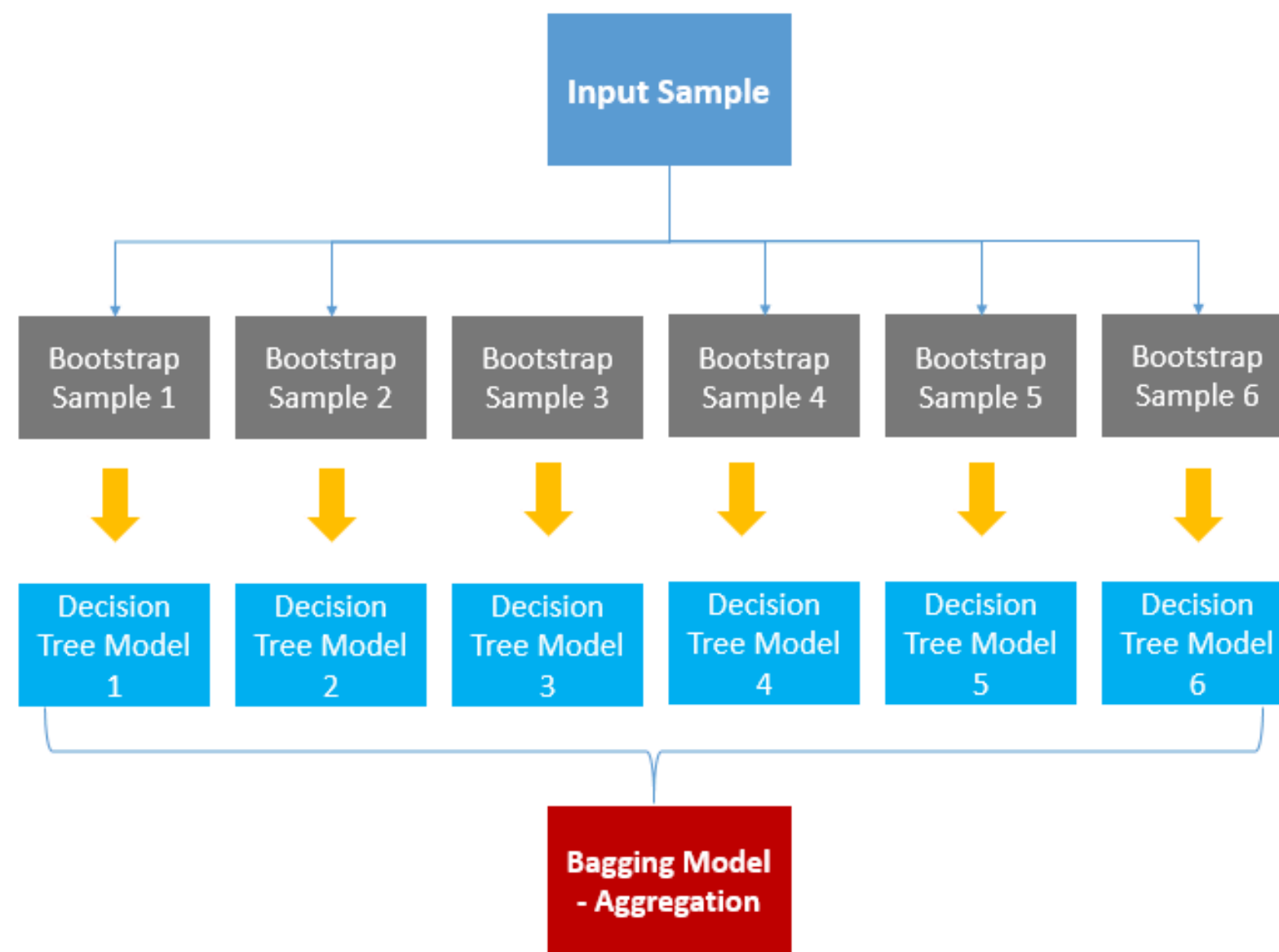
Bagging – Bootstrap

- Sample size가 N , replication 수를 B 라고 하자.
 - sample에서 N 개를 복원 추출 (중복된 값 허용)
 - B 번 반복
 - B 개의 sample을 통해 통계량 계산



Bagging

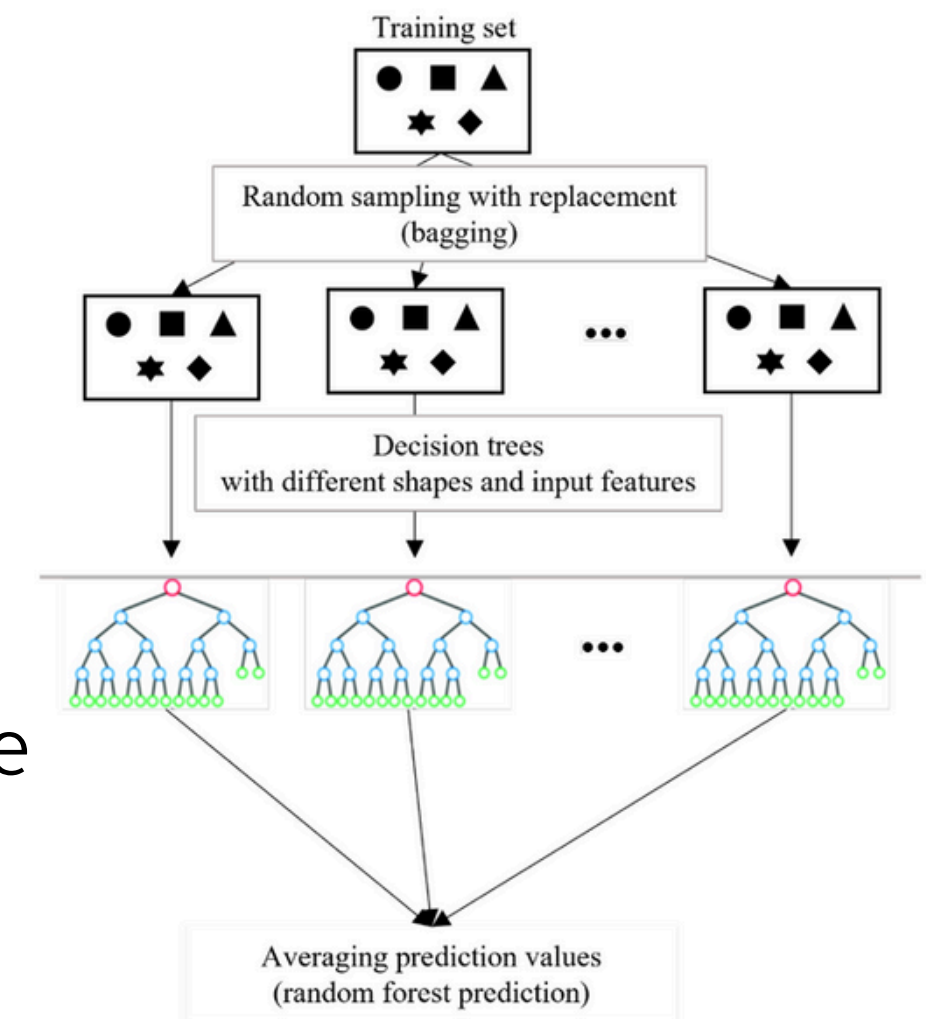
- **Bagging = Bootstrap + Aggregating(Average)**
- Bootstrap : sampling with Replacement → Variance 개선



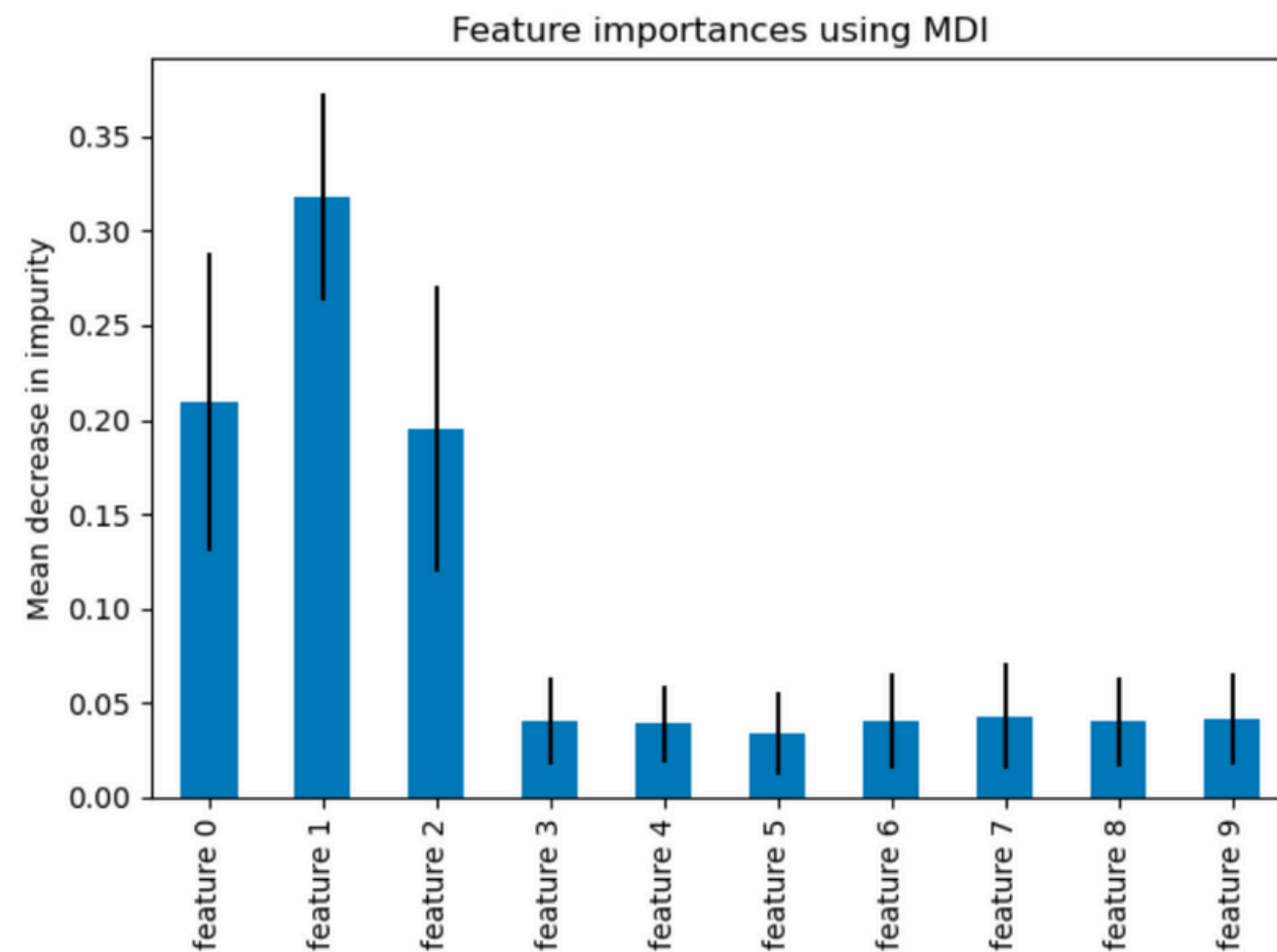
문제은행!

Bagging – Random Forest

- **Feature Bagging → RandomForest**
- Random Forest Decision Tree Generation
- Forest-RI (random input)
 - randomly select F features to split each node of the decision tree
- Forest-RC (randomly combined)
 - F randomly combined new features (F linear combination)
- Randomly select
 - one of the F best splits at each node of decision tree



Bagging – Random Forest



- Random Forest는 Bootstrap Sampling을 통해 여러 트리를 생성
 - 각 트리는 무작위로 선택된 하위 특성(subset of features)만을 사용해 학습되므로
- 어떤 특성이 여러 트리에서 일관되게 중요한 역할을 한다면, 전체 모델에서도 중요한 특성으로 간주
- **feature importance를 구할 수 있음!**

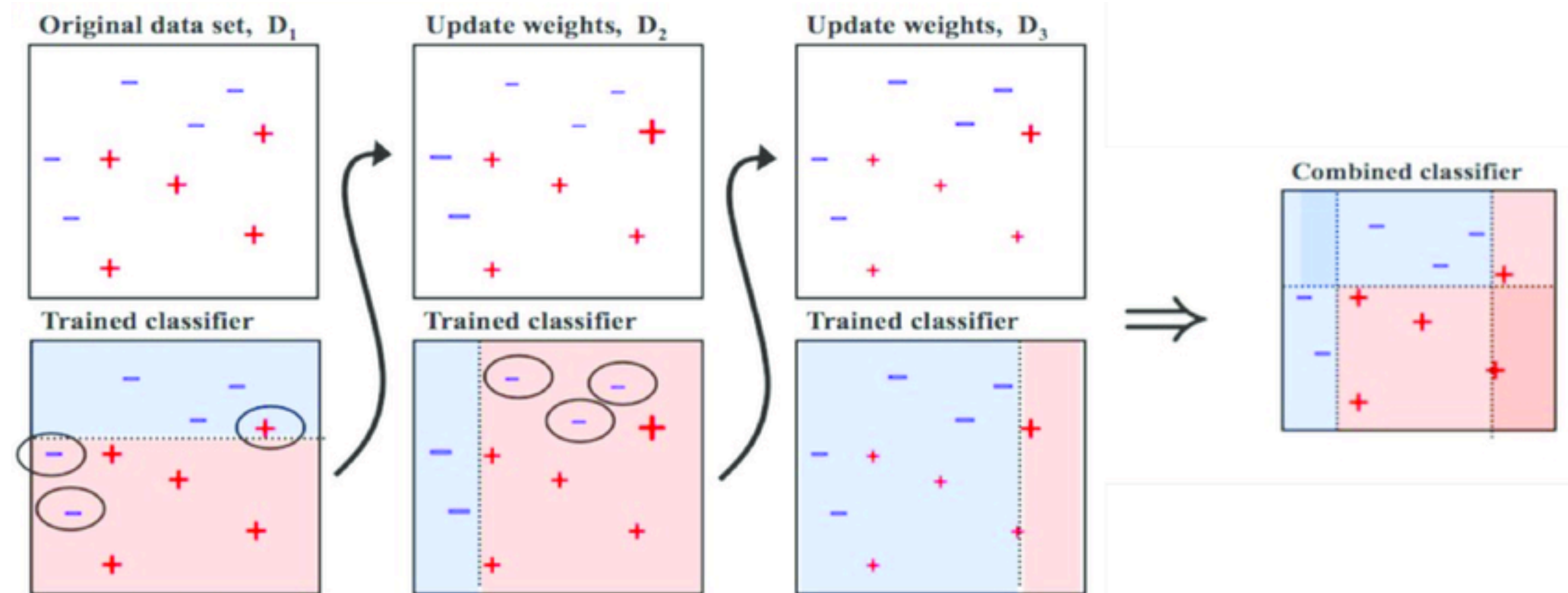
Boosting

• Boosting

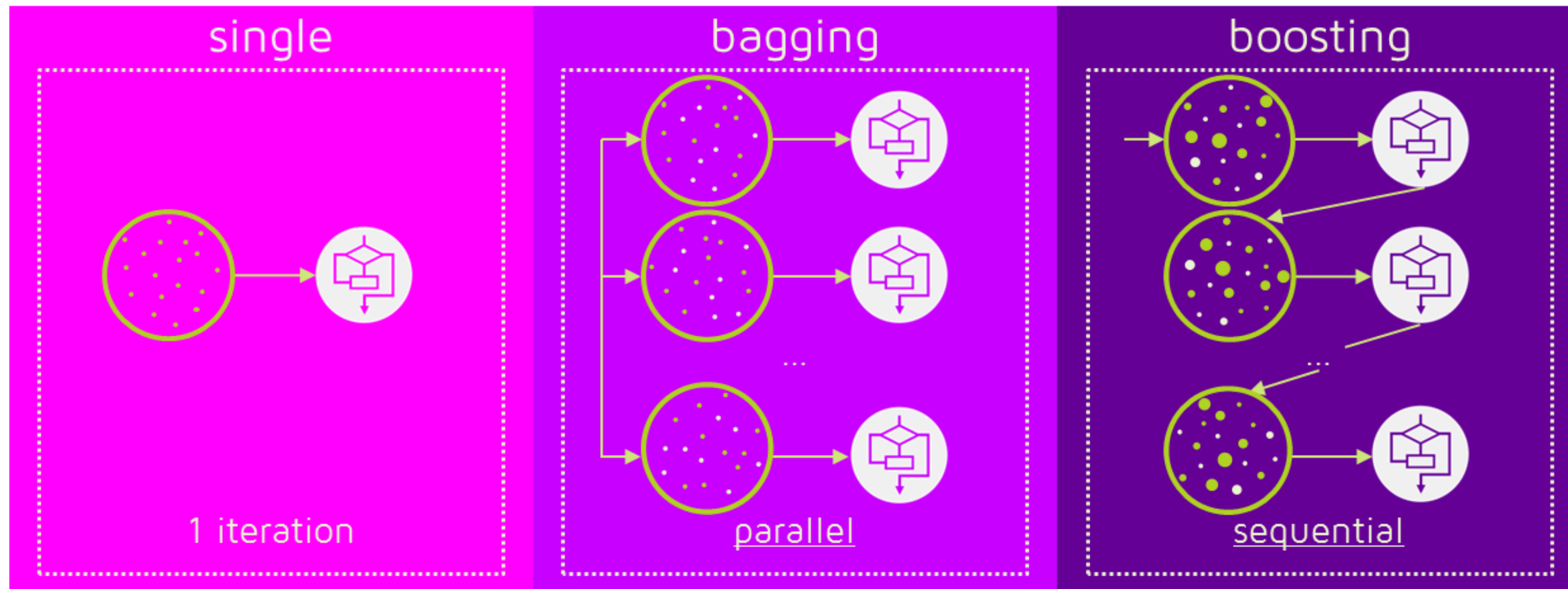
- 기존 모델이 틀린 샘플에 대해 더 많은 가중치를 부여하며 학습하는 기법
 - 이전 모델이 잘못 예측한 부분을 집중적으로 학습
- 예측이 틀린 데이터가 다시 뽑힐 가중치가 높아짐
- 여러 개의 Weak Learner를 순차적으로 학습하며 점진적으로 개선

오답노트!

**Bias 개선,
BUT 시간이 오래 걸림
Bootstrapping 사용 X
→ Sequential 하게 학습**



Bagging vs Boosting



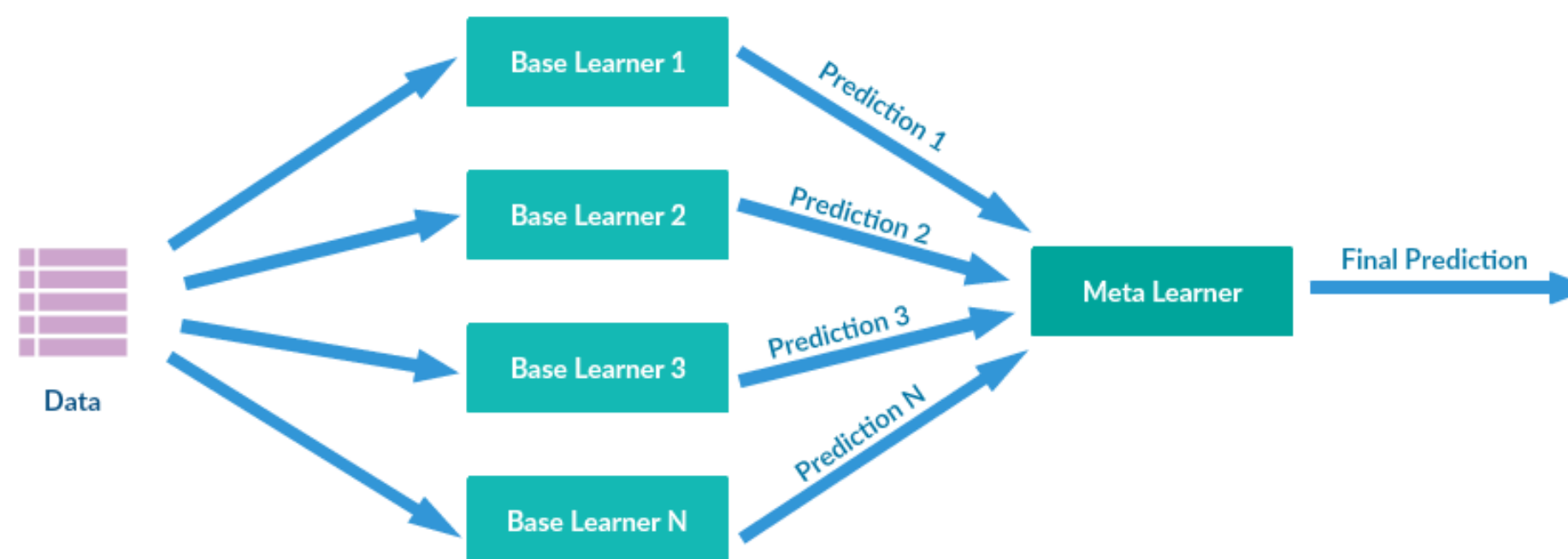
Stacking

- **Stacking Generalization**

- Meta-learning model
- 여러 가지 모델들의 예측값을 최종 모델의 학습 데이터로 사용

- **K-fold cv**

- Step 0 : 각 weak model에 k-fold CV를 적용하여 예측 데이터를 형성
- Step 1 : step 0에서 만든 예측 데이터를 stack하여 meta-model을 train 및 예측





해당 자료는 Slack & KUBIG github에서 보실 수 있습니다 :)

6주차 과제는 2/18 수요일까지 github에 업로드 해 주시기 바랍니다!

6주차 과제 : KUBIG_2026_SPRING/Basic Study/ML/6주차
파일명 2026_1_ML_week6_{본인 이름}.ipynb 로 제출